
Spring Boot



Testing Spring Boot Applications Demystified

The Hero's Journey Through the Spring Boot Testing Labyrinth

Java Guild@Elia · October 20, 2026

Philip Riecks · PragmaTech GmbH · @rieckpil

About Philip

- Self-employed developer from Herzogenaurach, now living in Erlangen, Germany (Bavaria) 🍻
- Blogging & content creation with a focus on testing Java and specifically Spring Boot applications 🍃
- Founder of PragmaTech GmbH - We **Enable Developers to Frequently Deliver** Software with **More Confidence** 🚤



Herzogenaurach, Bavaria

Participate During the Talk

Go to menti.com and use the code **7854 5371** to **anonymously** submit answers for the quizzes and add your questions during the talk.



Please answer the questions, we'll wait a bit to continue.

Why Test Software?



In 2026, most of our code is not written by us.

Agentic Development Today

- AI makes software faster to create and harder to trust.
- AI made writing code cheap. Verifying it is the new bottleneck.
- AI adoption correlates with increased throughput and decreased stability.
- AI generates the logic, but you inherit the **liability**. It can write the function; it won't join the post-mortem.

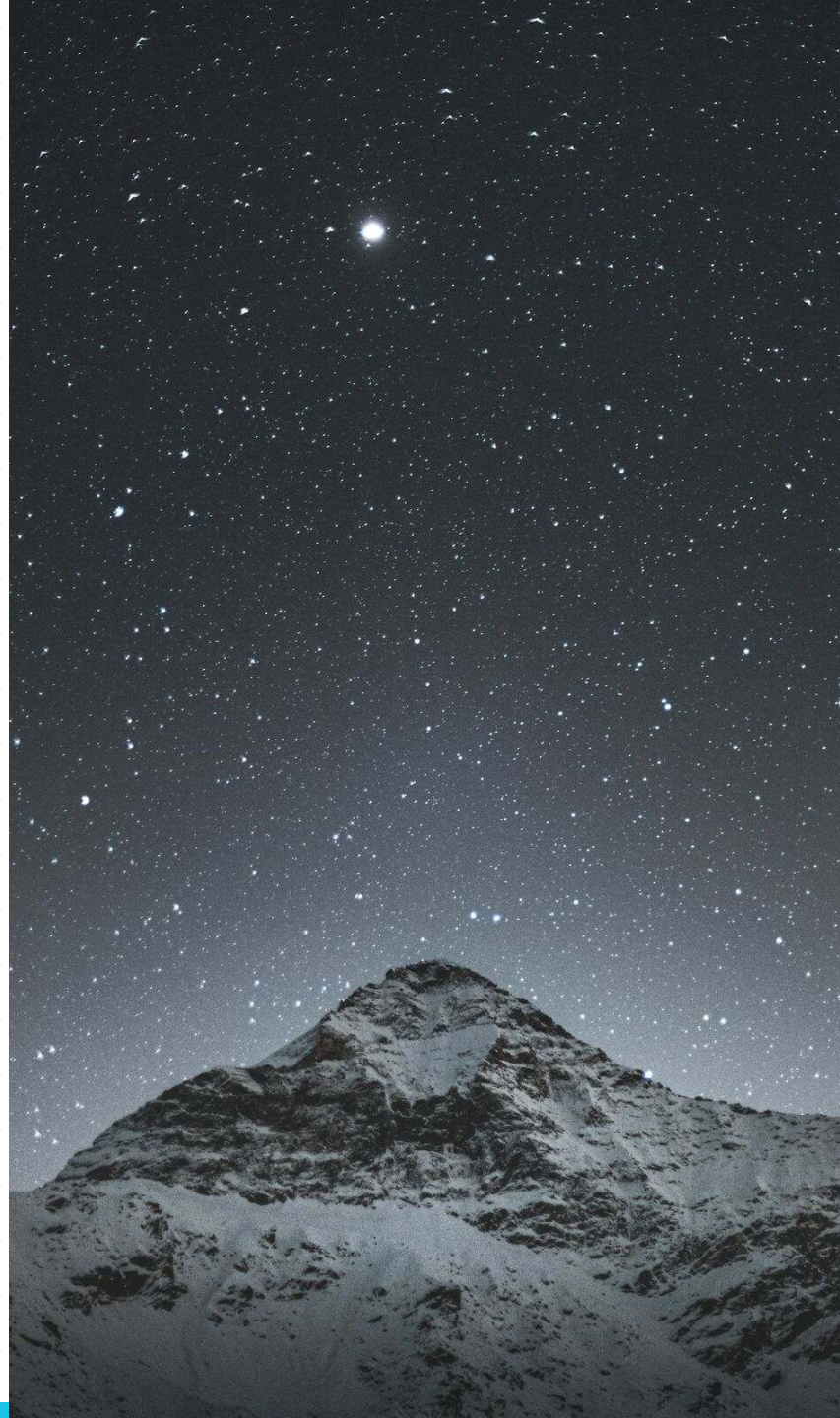


My Overall Northstar for Engineering Excellence

Imagine seeing this pull request on a Friday afternoon:



How **confident** are you to merge this major Spring Boot upgrade and **deploy** it to production once the pipeline turns green?



Good tests don’t just catch bugs - they give you fast feedback and confident deployments.



How do we get there for our Spring Boot projects?



The Hero’s Journey aka. Our Agenda



Goals For This Talk

- **Provide a clear mental map** for choosing between unit, slice, and integration tests
- Get **better judgment** for AI-written tests
- Understand the **testing concepts** to better reason about a test strategy or test failures
- **Build confidence** into your development work to **ship fearlessly**





Act 1: The Grand Entrance

Testing Spring Boot applications can feel like entering a labyrinth blindfolded:

- Copying test configuration from AI/StackOverflow hoping it works
- The paradox of choice: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@MockBean`, etc.
- Struggling with Spring `ApplicationContext` creation during tests
- Uncertainty about what to test and how to test it effectively

QUEST 1

The Unit Testing Guardian

The Swift Gatekeeper - Blocks Those Who Overcomplicate



Our Foundation: Spring Boot Starter Test

- The "Testing Swiss Army Knife"

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

- Batteries-included for testing by transitively including popular testing libraries
- Out-of-the-box dependency management to ensure compatibility



```
./mvnw dependency:tree
[INFO] ...
[INFO] +- org.springframework.boot:spring-boot-starter-test:jar:4.0.2:test
[INFO] |  +- org.springframework.boot:spring-boot-test:jar:4.0.2:test
[INFO] |  +- org.springframework.boot:spring-boot-test-autoconfigure:jar:4.0.2:test
[INFO] |  +- com.jayway.jsonpath:json-path:jar:2.10.0:test
[INFO] |  |  \- org.slf4j:slf4j-api:jar:2.0.17:compile
[INFO] |  +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.4:test
[INFO] |  |  \- jakarta.activation:jakarta.activation-api:jar:2.1.4:test
[INFO] |  +- net.minidev:json-smart:jar:2.6.0:test
[INFO] |  |  \- net.minidev:accessors-smart:jar:2.6.0:test
[INFO] |  |     \- org.ow2.asm:asm:jar:9.7.1:test
[INFO] |  +- org.assertj:assertj-core:jar:3.27.6:test
[INFO] |  |  \- net.bytebuddy:byte-buddy:jar:1.17.8:test
[INFO] |  +- org.awaitility:awaitility:jar:4.3.0:test
[INFO] |  +- org.hamcrest:hamcrest:jar:3.0:test
[INFO] |  +- org.junit.jupiter:junit-jupiter:jar:6.0.2:test
[INFO] |  |  +- org.junit.jupiter:junit-jupiter-api:jar:6.0.2:test
[INFO] |  |  |  +- org.opentest4j:opentest4j:jar:1.3.0:test
[INFO] |  |  |  +- org.junit.platform:junit-platform-commons:jar:6.0.2:test
[INFO] |  |  |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO] |  |  +- org.junit.jupiter:junit-jupiter-params:jar:6.0.2:test
[INFO] |  |  \- org.junit.jupiter:junit-jupiter-engine:jar:6.0.2:test
[INFO] |  |     \- org.junit.platform:junit-platform-engine:jar:6.0.2:test
[INFO] |  +- org.mockito:mockito-core:jar:5.5.0:test
[INFO] |  |  +- net.bytebuddy:byte-buddy-agent:jar:1.17.8:test
[INFO] |  |  \- org.objenesis:objenesis:jar:3.3:test
[INFO] |  +- org.mockito:mockito-junit-jupiter:jar:5.5.0:test
[INFO] |  +- org.skyscreamer:jsonassert:jar:1.5.3:test
[INFO] |  |  \- com.vaadin.external.google:android-json:jar:0.0.20131108.vaadin1:test
[INFO] |  +- org.springframework:spring-core:jar:7.0.3:compile
[INFO] |  |  +- commons-logging:commons-logging:jar:1.3.5:compile
[INFO] |  |  \- org.jspecify:jspecify:jar:1.0.0:compile
[INFO] |  +- org.springframework:spring-test:jar:7.0.3:test
[INFO] |  \- org.xmlunit:xmlunit-core:jar:2.10.4:test
```

What’s Inside the Testing Swiss Army Knife?

- **JUnit**: Java's de-facto standard testing framework and foundation.
- **Mockito**: Creating mock objects to simulate dependencies and verify interactions.
- **AssertJ**: Provides fluent, chainable, and readable assertions.
- **Hamcrest**: Offers flexible matchers for creating custom assertions.
- **JSONAssert**: Compares JSON strings with flexible matching options.
- **JsonPath**: Extracts and queries data from JSON similar to XPath.
- **XMLUnit**: Compares and validates XML documents.
- **Awaitility**: Handles asynchronous testing with fluent conditions.

Unit Testing Java/Spring Boot Applications 101

- **Core Concept**: Test individual components in isolation from dependencies - one unit of work at a time.
- **Confidence Gained**: Fast, high-volume verification that the smallest building blocks behave correctly under various conditions.
- **Pitfall**: Poor class design leads to untestable god classes. Good tests start with good design.
- **Tools**: JUnit, Mockito, AssertJ (or Spock, TestNG, Hamcrest).

Unit Testing has Limits

Consider this sample REST controller, what could we verify with a unit test?

```java
@RestController
@RequestMapping("/api/customers")
public class CustomerController {

  private final CustomerService customerService;

  public CustomerController(CustomerService customerService) {
    this.customerService = customerService;
  }

  @PostMapping
  public ResponseEntity<Void> createNewCustomer(@Validated CustomerCreationRequest payload, UriComponentsBuilder uriBuilder) {

    String customerId = customerService.createNewCustomer(payload.firstName());

    UriComponents uriComponents = uriBuilder
      .path("/api/customers/{id}")
      .buildAndExpand(customerId);

    return ResponseEntity.created(uriComponents.toUri()).build();
  }
}
```

```java
@ExtendWith(MockitoExtension.class)
class CustomerControllerUnitTests {

  @Mock
  private CustomerService customerService;

  @InjectMocks
  private CustomerController customerController;

  @Test
  void shouldCreateCustomerWhenPayloadRequestIsValid() {
    when(customerService.createNewCustomer(anyString()))
      .thenReturn("42");

    ResponseEntity<Void> result = customerController.createNewCustomer(
      new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
      UriComponentsBuilder.newInstance()
    );

    assertThat(result.getStatusCode().value())
      .isEqualTo(201);
    assertThat(result.getHeaders().getLocation().toString())
      .isEqualTo("/api/customers/42");
  }
}
```



Things We Can't Cover With a Unit Test #0

Request Mapping: Does HTTP GET `/api/customers/{id}` actually resolve to our desired method?

```
1   @Test
2   void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7       new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
8       UriComponentsBuilder.newInstance()
9     );
10  }
```

Things We Can’t Cover With a Unit Test #1

Validation: Will an incomplete request body result in a 400 bad request or return an accidental 201?

```
1   @Test
2   void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7       new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8       UriComponentsBuilder.newInstance()
9     );
10  }
```

Things We Can't Cover With a Unit Test #2

Serialization: Are we JSON objects serialized and deserialized correctly?

```
ResponseEntity<Void> result = customerController.createNewCustomer(
  new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
  UriComponentsBuilder.newInstance());
```

```
{
  "first-name": "Java",
  "last_Name": "Duke",
  "email": "duke@spring.io"
}
```

Things We Can't Cover With a Unit Test #3

Security: Are we Spring Security configuration and other authorization checks enforced?

```
1   @Test
2   void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7       new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8       UriComponentsBuilder.newInstance()
9     );
10  }
```

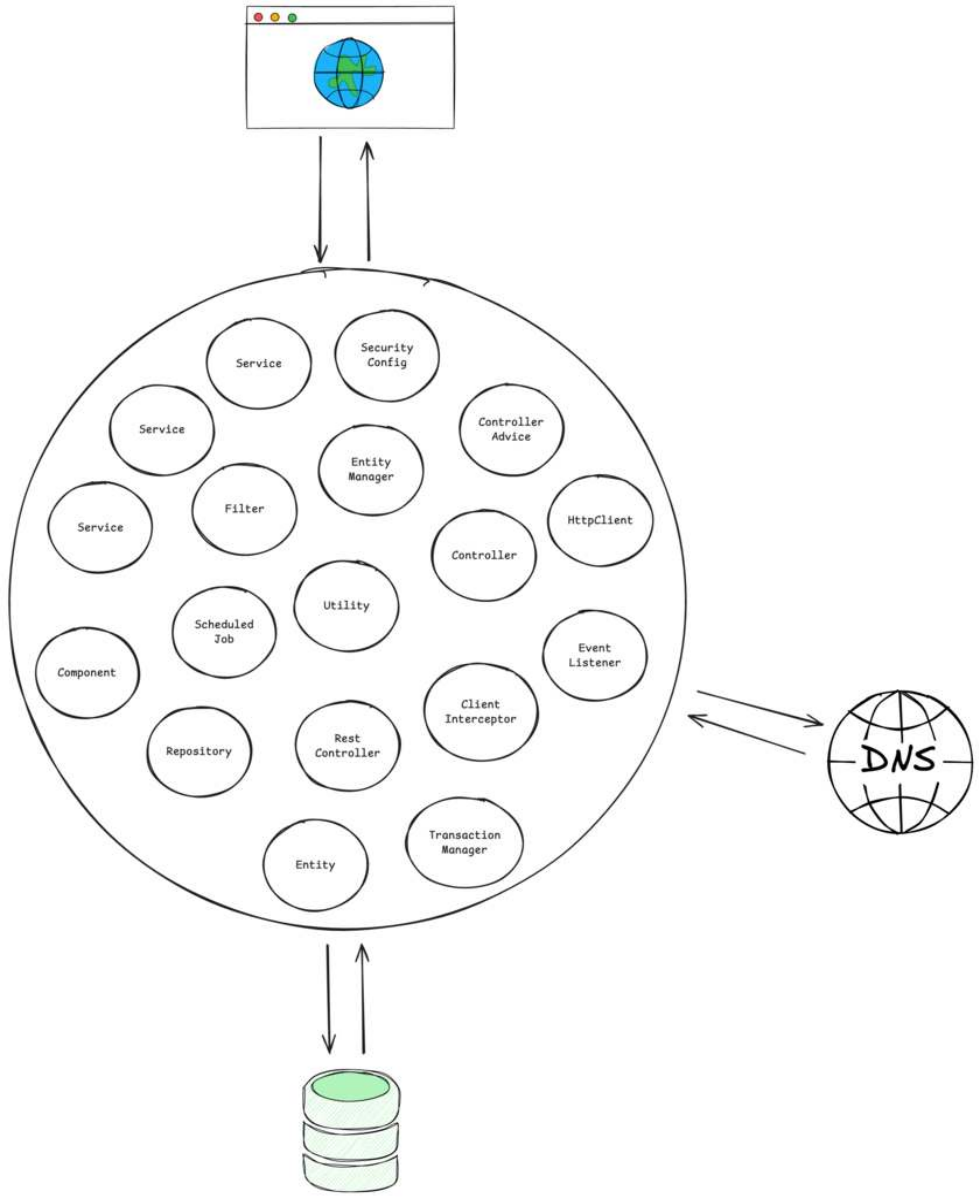
QUEST 2

The Slice Testing Hydra

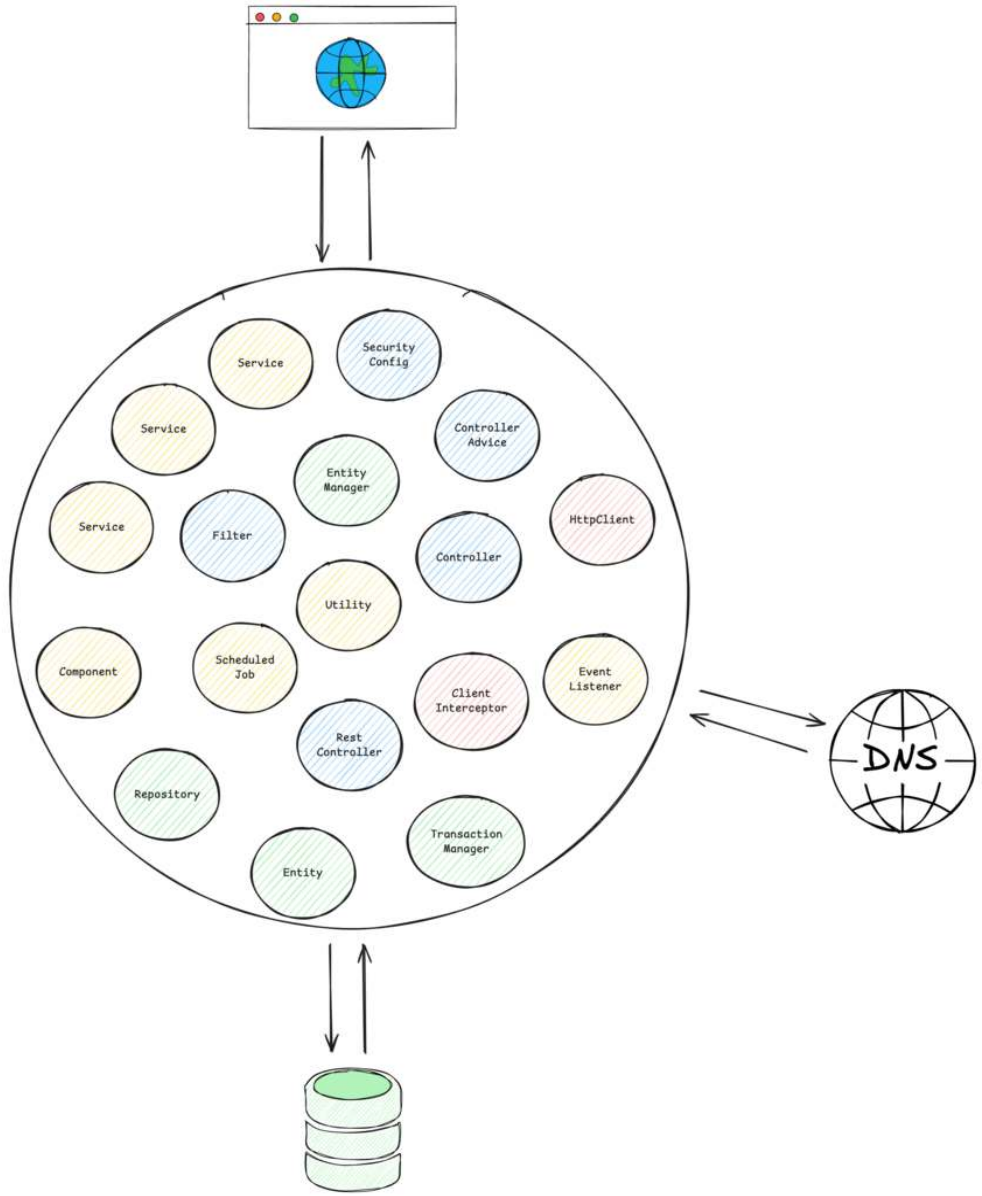
Multiple Heads, Each Guarding a Layer

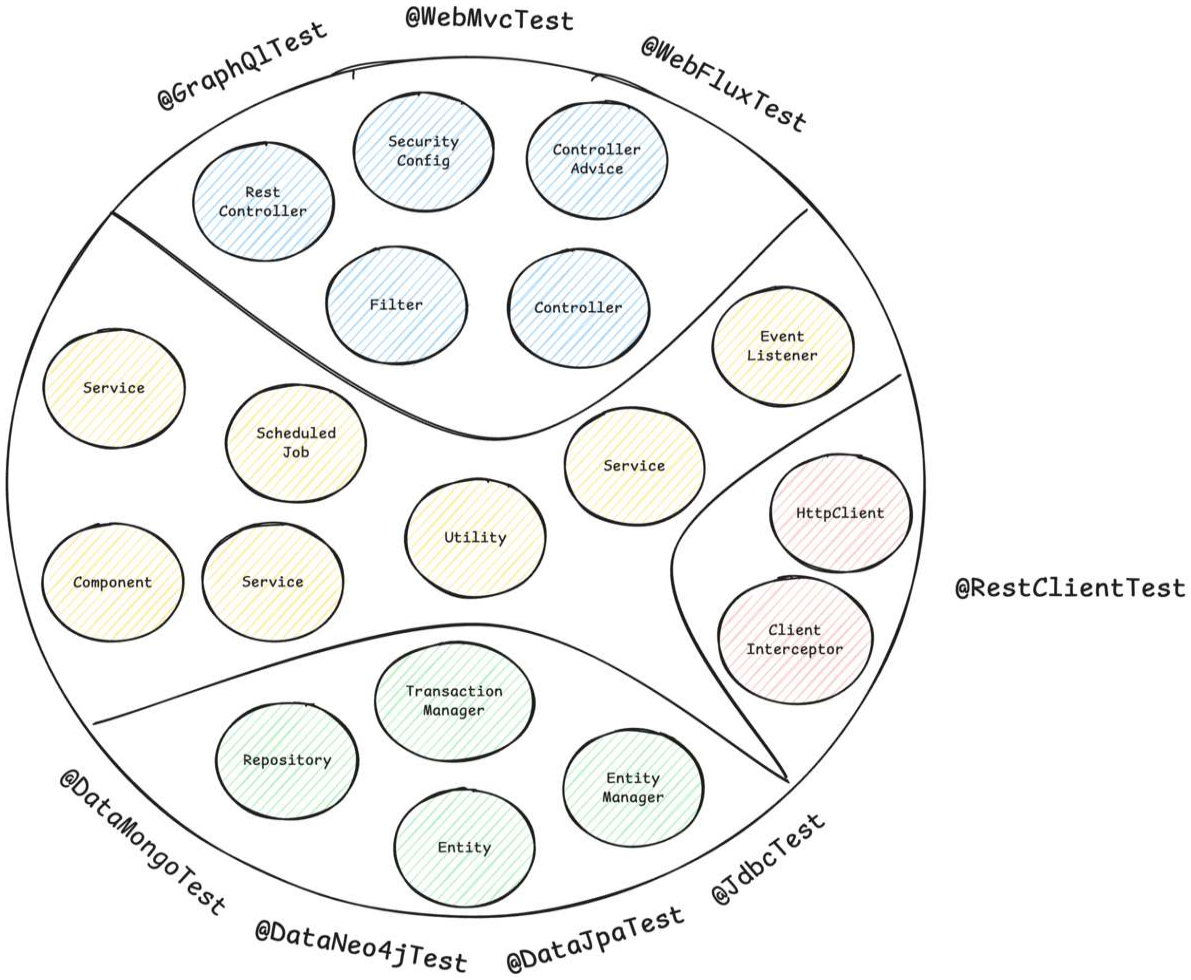


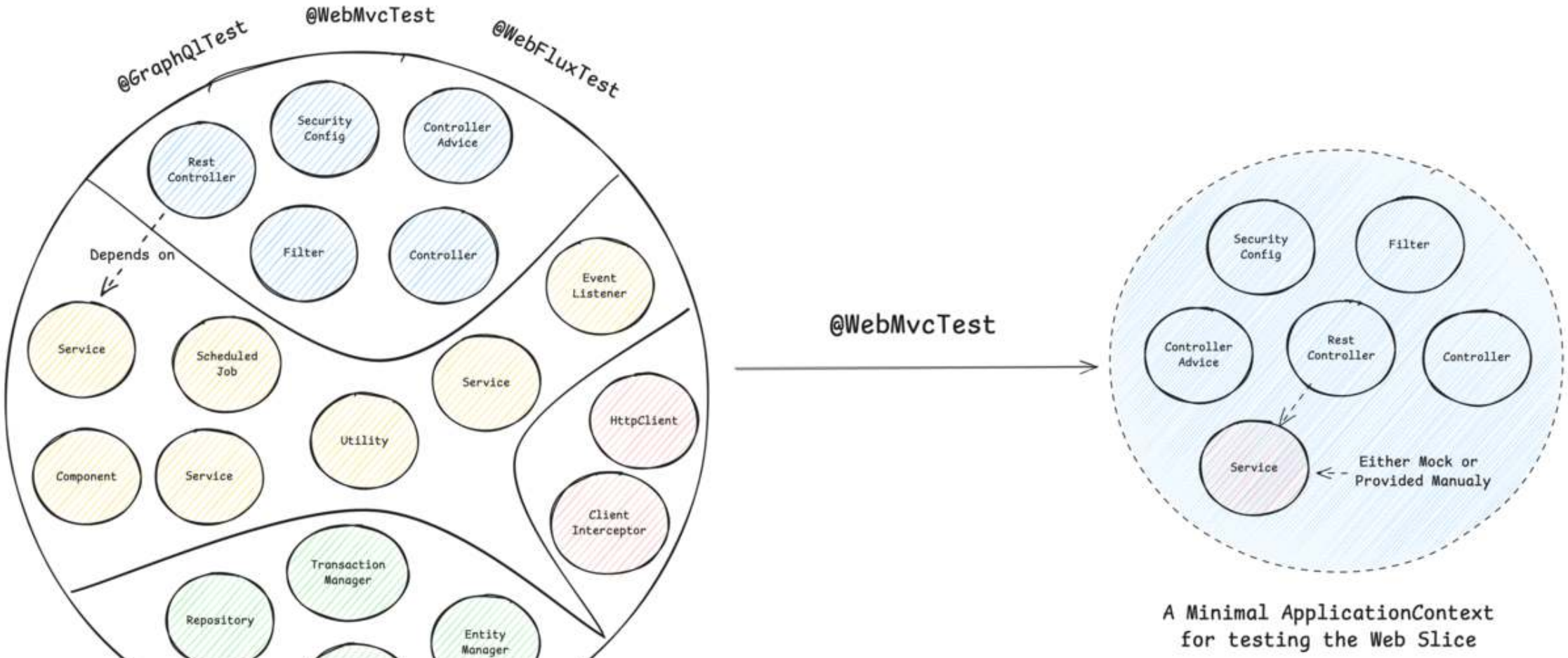
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext




@GraphQlTest
@WebMvcTest
@WebFluxTest
Security Config
Controller Advice
Rest Controller
Filter
Controller
Event Listener
Service
Scheduled Job
Service
HttpClient
Utility
Component
Service
Client Interceptor
@RestClientTest
Transaction Manager
Repository
Entity Manager
Entity
@DataMongoTest
@DataNeo4jTest
@DataJpaTest
@JdbcTest


@GraphQlTest
@WebMvcTest
@WebFluxTest
Security Config
Controller Advice
Rest Controller
Filter
Controller
Depends on
Event Listener
Service
Scheduled Job
Service
HttpClient
Utility
Component
Service
Client Interceptor
Transaction Manager
Repository
Entity Manager
Entity
@WebMvcTest
Security Config
Filter
Controller Advice
Rest Controller
Controller
Service
Either Mock or Provided Manualy
A Minimal ApplicationContext for testing the Web Slice

Spring Boot Test Slice Example: `@WebMvcTest`

```java
@WebMvcTest(CustomerController.class)
@Import(SecurityConfig.class)
class CustomerControllerTest {

  @Autowired
  private MockMvc mockMvc;

  @MockitoBean
  private CustomerService customerService;

  @Test
  @WithMockUser
  void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
    // ...
  }
}
```

Common Test Slices

- `@WebMvcTest` / `@WebFluxTest` - Controller layer
- `@DataJpaTest` / `@JdbcTest` - Persistence layer
- `@JsonTest` - JSON serialization/deserialization
- `@RestClientTest` - RestTemplate testing
- etc.


@DataCouchbaseTest
@DataLdapTest
@RestClientTest
@JsonTest
@JooqTest
@WebFluxTest
@DataRedisTest
@WebMvcTest
@DataMongoTest
@DataCassandraTest
@GraphQlTest
@DataNeo4jTest
@WriteYourOwn*
@DataJpaTest
@SqsTest
@JdbcTest
@DataElasticsearchTest

Sliced Testing Spring Boot Applications 101

- **Core Concept**: Test a specific "slice" or layer of your application by loading a minimal, relevant part of the Spring `ApplicationContext`.
- **Confidence Gained**: Helps validate parts of your application where pure unit testing is insufficient, like the web, messaging, or data layer.
- **Prominent Examples:** Web layer (`@WebMvcTest`) and database layer (`@DataJpaTest`)
- **Pitfalls**: Requires careful configuration to ensure only the necessary slice of the context is loaded.
- **Tools**: JUnit, Mockito, Spring Test, Spring Boot, Testcontainers

QUEST 3

The Integration Testing Dragon

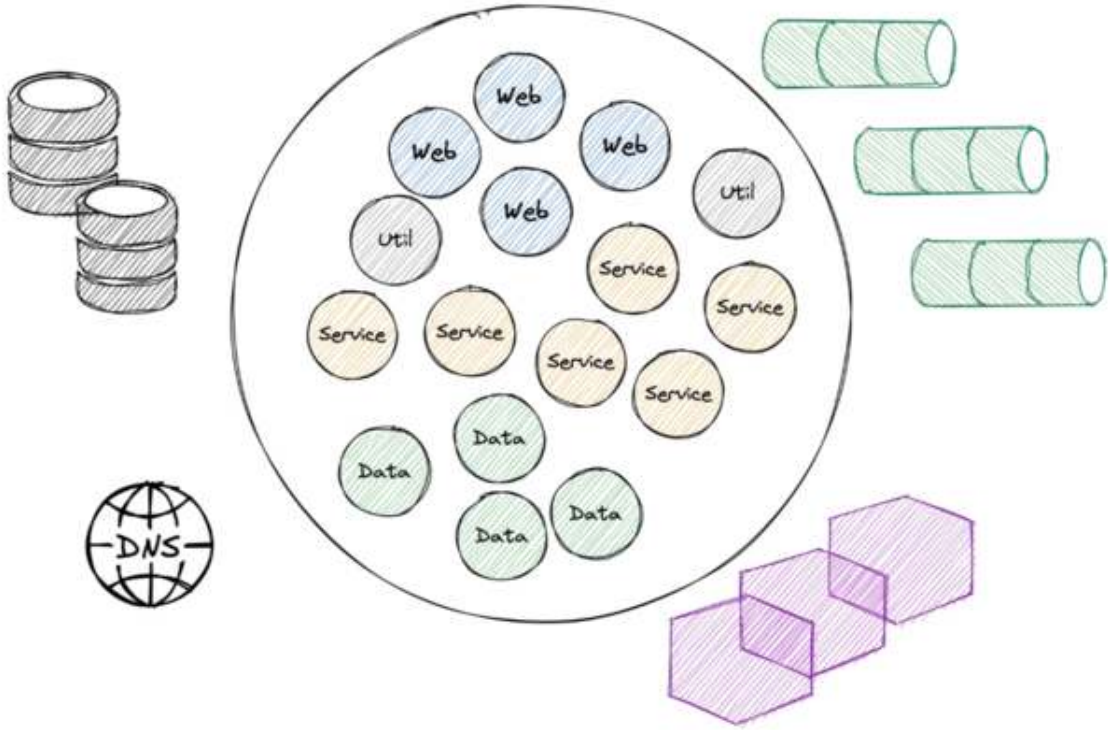
Guards the Full Treasure - but Demands Patience



```java
@SpringBootTest
class ApplicationTest {

  @Test
  void contextLoads() {
  }

}
```



Challenges when Starting the Entire `ApplicationContext`

- **Problem #1**: How to Ensure Surrounding Infrastructure (e.g. database, queues, etc.) is Present?
- **Problem #2**: How to Interact with our Application for Integration Tests?
- **Problem #3**: How to Keep our Build Time at a reasonable Duration?

There's Even More...

- **Problem #4**: How to handle HTTP communication from our application to remote services?
- **Problem #5**: How to provide test data and maintain a clean state between tests?
- **Problem #6**: How to handle authentication and security contexts during tests?

Provide External Infrastructure with Testcontainers (Problem #1)

Running infrastructure components (databases, message brokers, etc.) in Docker containers for our tests becomes a breeze with Testcontainers:

```
@Container
@ServiceConnection
static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:16-alpine")
  .withDatabaseName("testdb")
  .withUsername("test")
  .withPassword("test")
  .withInitScript("init-postgres.sql");
```

This gives us an ephemeral PostgreSQL database for our tests:

```
$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS                                                NAMES
a958ee2887c6   postgres:16-alpine             "docker-entrypoint.s…"   10 seconds ago   Up 9 seconds    0.0.0.0:32776->5432/tcp, [::]:32776->5432/tcp        affectionate_cannon
ad0f804068dc   testcontainers/ryuk:0.12.0     "/bin/ryuk"              10 seconds ago   Up 9 seconds    0.0.0.0:32775->8080/tcp, [::]:32775->8080/tcp        testcontainers-ryuk-1f9f76a6-46d4-4e19-85c1-e8364da12804
```

How to Interact with our Application for Integration Tests?

Option #1:

```
1   @SpringBootTest
2   // which is @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
3   @AutoConfigureMockMvc
4   class ApplicationMockWebIT {
5
6     // @LocalServerPort
7     // private int port; <-- this would fail the test, there is no local port occupied
8
9     @Test
10    @WithMockUser
11    void givenCustomersThenReturnListForAuthenticatedUser(@Autowired MockMvc mockMvc) throws Exception {
12      mockMvc
13        .perform(get("/api/customers")
14          .header(ACCEPT, APPLICATION_JSON))
15        .andExpect(status().is(200))
16        .andExpect(content().contentType(APPLICATION_JSON))
17        .andExpect(jsonPath("$.size()", is(1)));
18    }
19  }
```

How to Interact with our Application for Integration Tests?

Option #2:

```
@AutoConfigureWebTestClient // required since Spring Boot 4
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class ApplicationServletContainerIT {

  @Autowired
  WebTestClient webTestClient;

  @Test
  void contextLoads() {
    webTestClient
      .get()
      .uri("/api/customers")
      .header("Authorization", "Basic " + Base64.getEncoder().encodeToString("user:dummy".getBytes()))
      .exchange()
      .expectStatus()
      .isOk();
  }
}
```

Integration Testing Spring Boot Applications 101

- **Core Concept**: Start the entire Spring application context, often on a random local port, and test the application through its external interfaces (e.g., REST API).
- **Confidence Gained**: Validates the integration of all internal components working together as a complete application.
- **Best Practices**: Use `@SpringBootTest` to run the app on a local port.
- **Pitfalls**: Slower to run than unit or sliced tests. Managing the lifecycle of dependent services can be complex.
- **Tools**: JUnit, Mockito, Spring Test, Spring Boot, Testcontainers, WireMock (for mocking external HTTP services), Selenium (for browser-based UI testing)



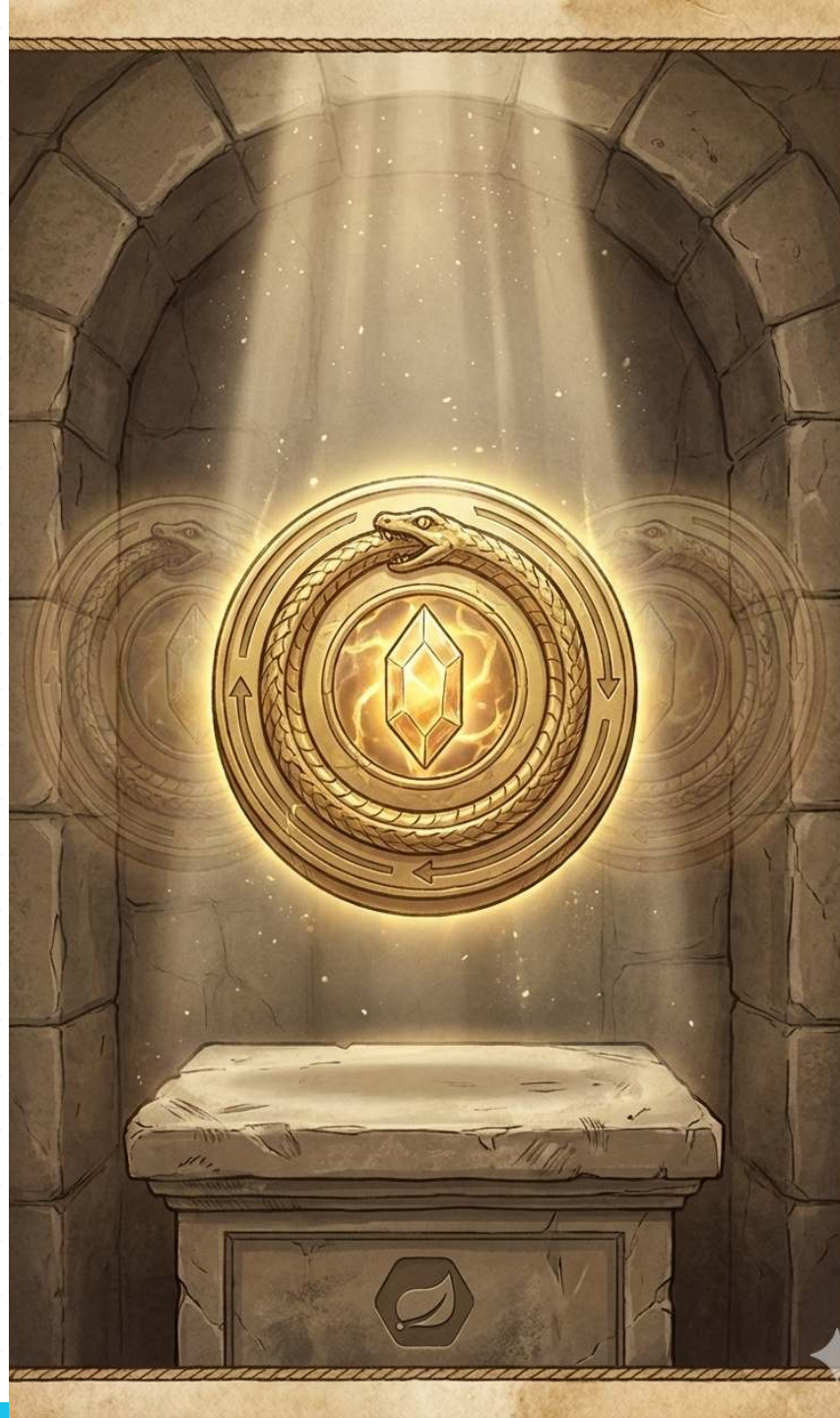
… but what about Problem #3: How to Keep our Build Time at a reasonable Duration?



QUEST ITEM 1

The Caching Amulet

Helps You Reuse What You Already Built



Integration Testing - The Need for Speed

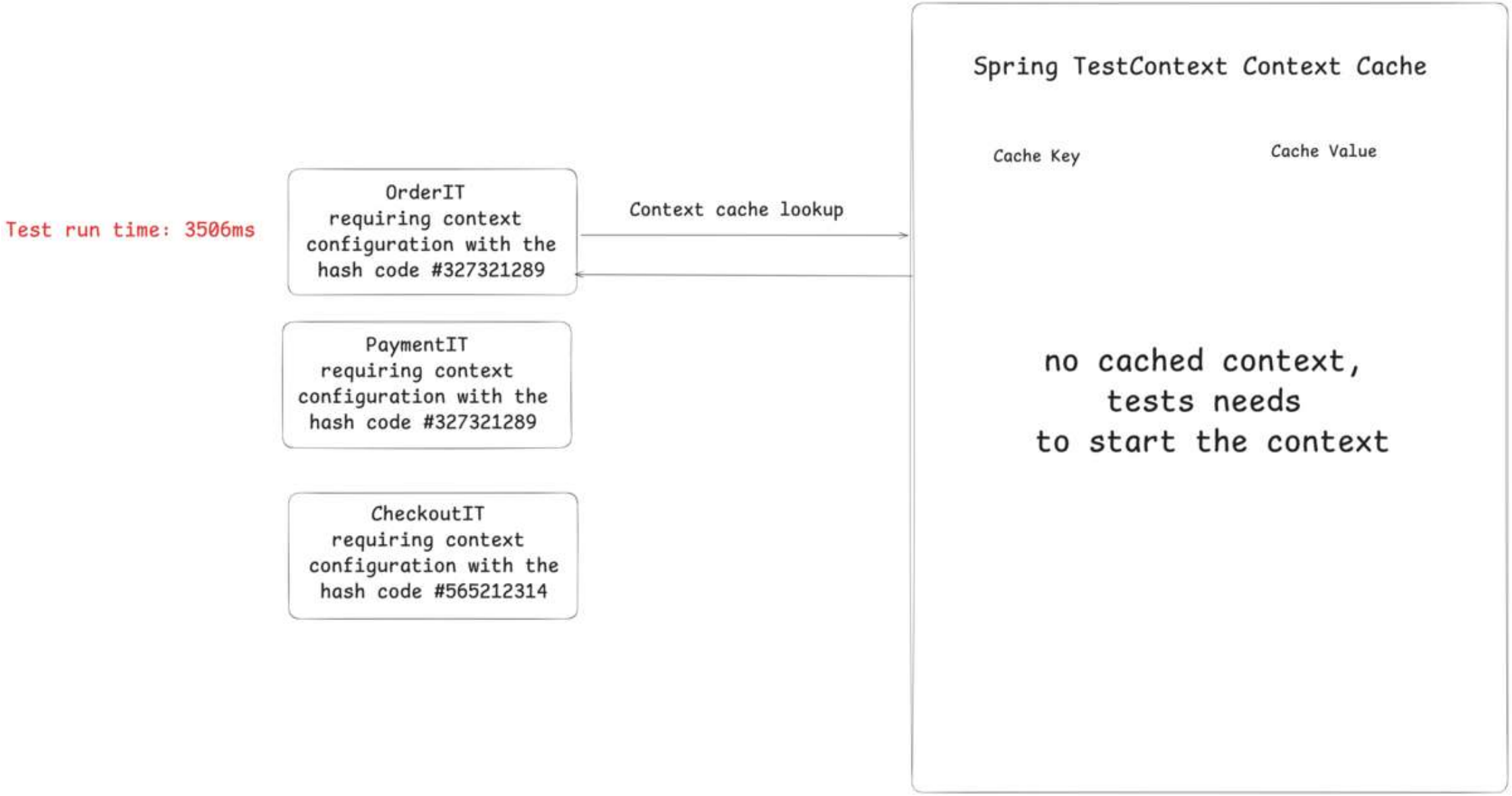
The Problem: Integration tests require a started & initialized Spring `ApplicationContext`, which slows down the build

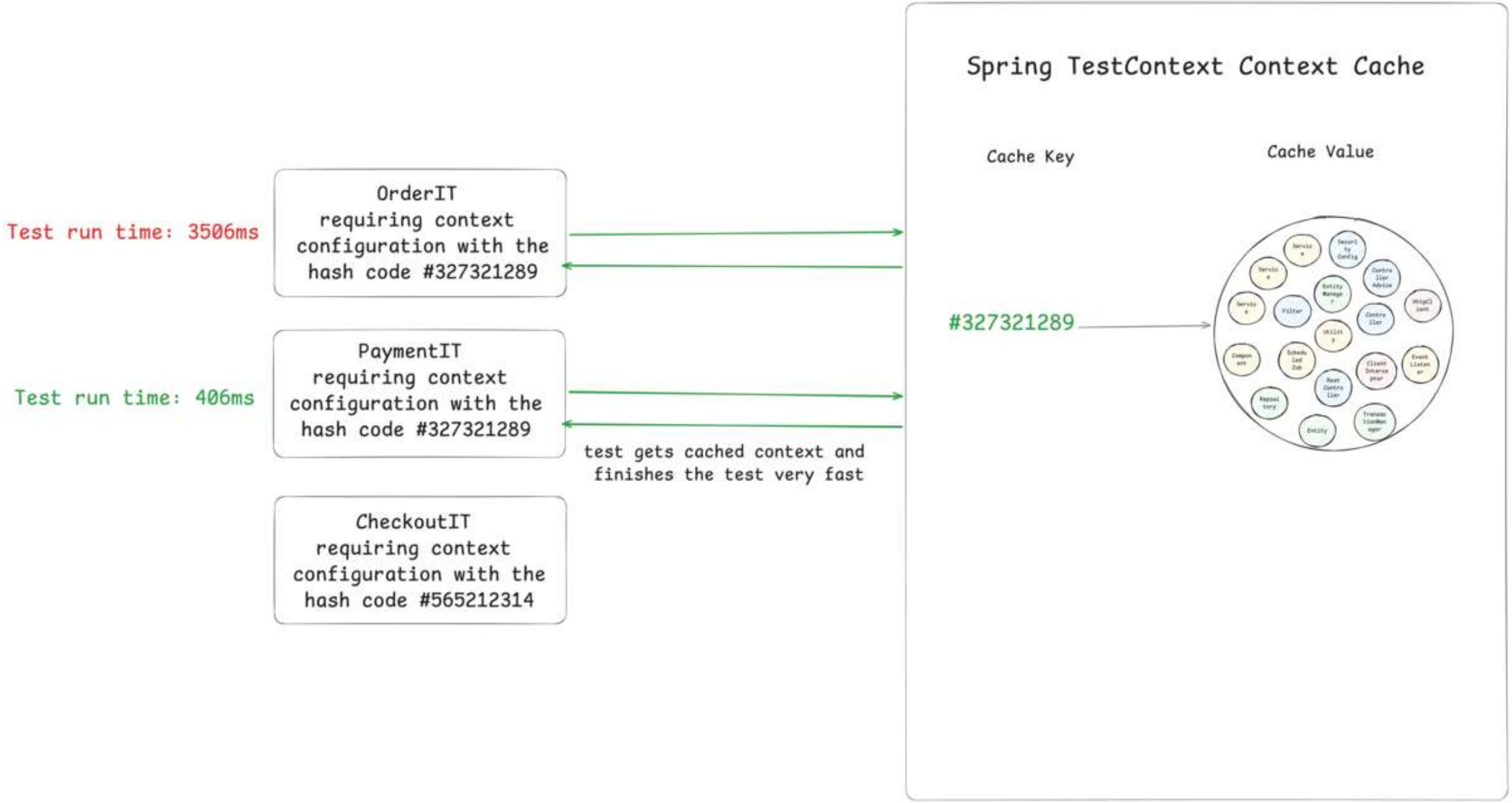
The Solution: Spring Test `TestContext` Caching - stores an already started Spring `ApplicationContext` for later reuse

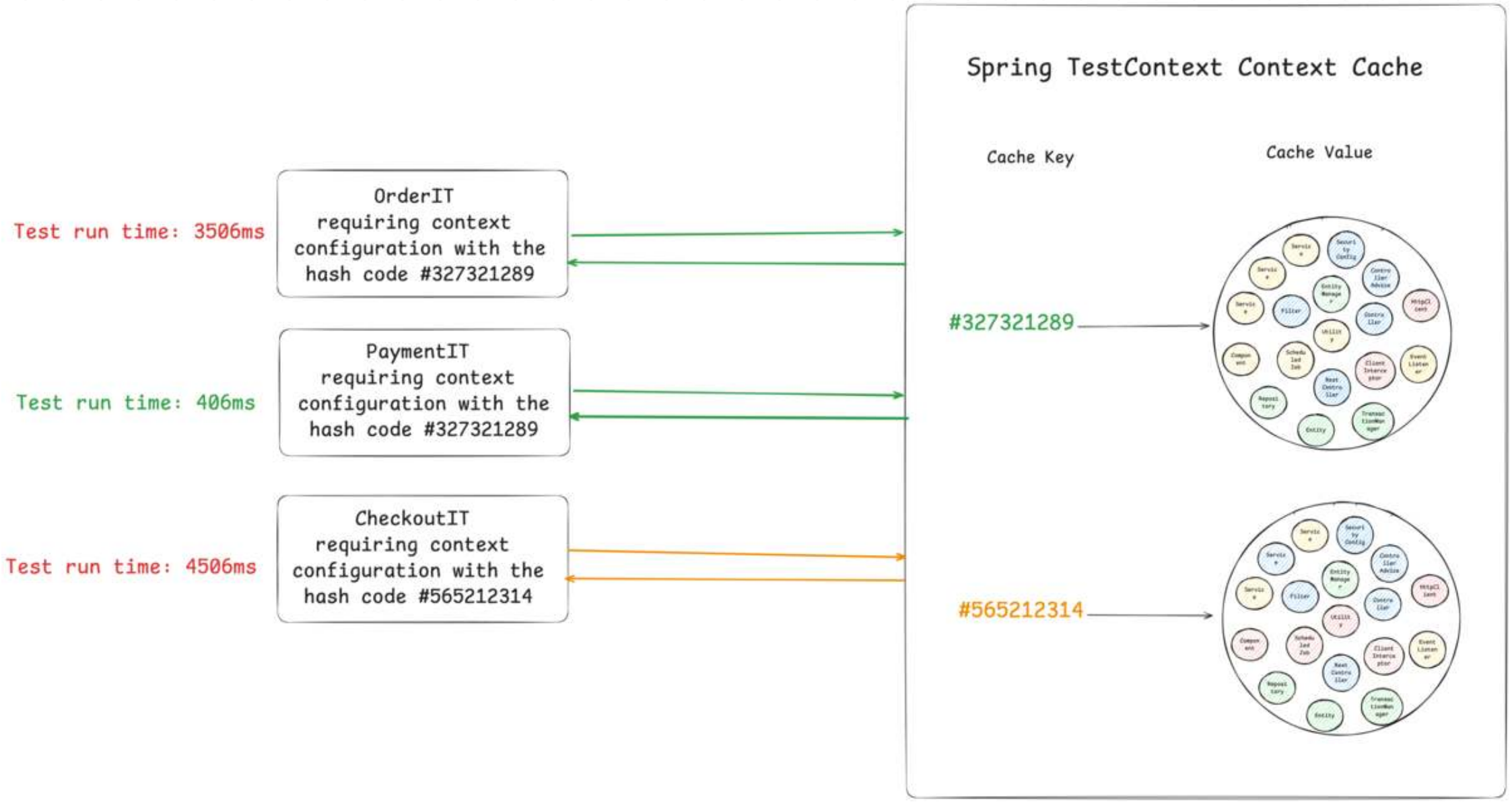
This feature is part of Spring Test (included in every Spring Boot project via `spring-boot-starter-test`)

Example of speed improvement:




Test run time: 3506ms
OrderIT
requiring context
configuration with the
hash code #327321289
Context cache lookup
PaymentIT
requiring context
configuration with the
hash code #327321289
CheckoutIT
requiring context
configuration with the
hash code #565212314
Spring TestContext Context Cache
Cache Key
Cache Value
no cached context,
tests needs
to start the context


Spring TestContext Context Cache
Cache Key
Cache Value
#327321289
Test run time: 3506ms
OrderIT
requiring context
configuration with the
hash code #327321289
Test run time: 406ms
PaymentIT
requiring context
configuration with the
hash code #327321289
test gets cached context and
finishes the test very fast
CheckoutIT
requiring context
configuration with the
hash code #565212314


Test run time: 3506ms
OrderIT
requiring context
configuration with the
hash code #327321289
Test run time: 406ms
PaymentIT
requiring context
configuration with the
hash code #327321289
Test run time: 4506ms
CheckoutIT
requiring context
configuration with the
hash code #565212314
Spring TestContext Context Cache
Cache Key
Cache Value
#327321289
#565212314

How the Cache Key is Built

```
1   // DefaultContextCache.java
2   private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LinkedHashMap<>(32, 0.75f, true));
```

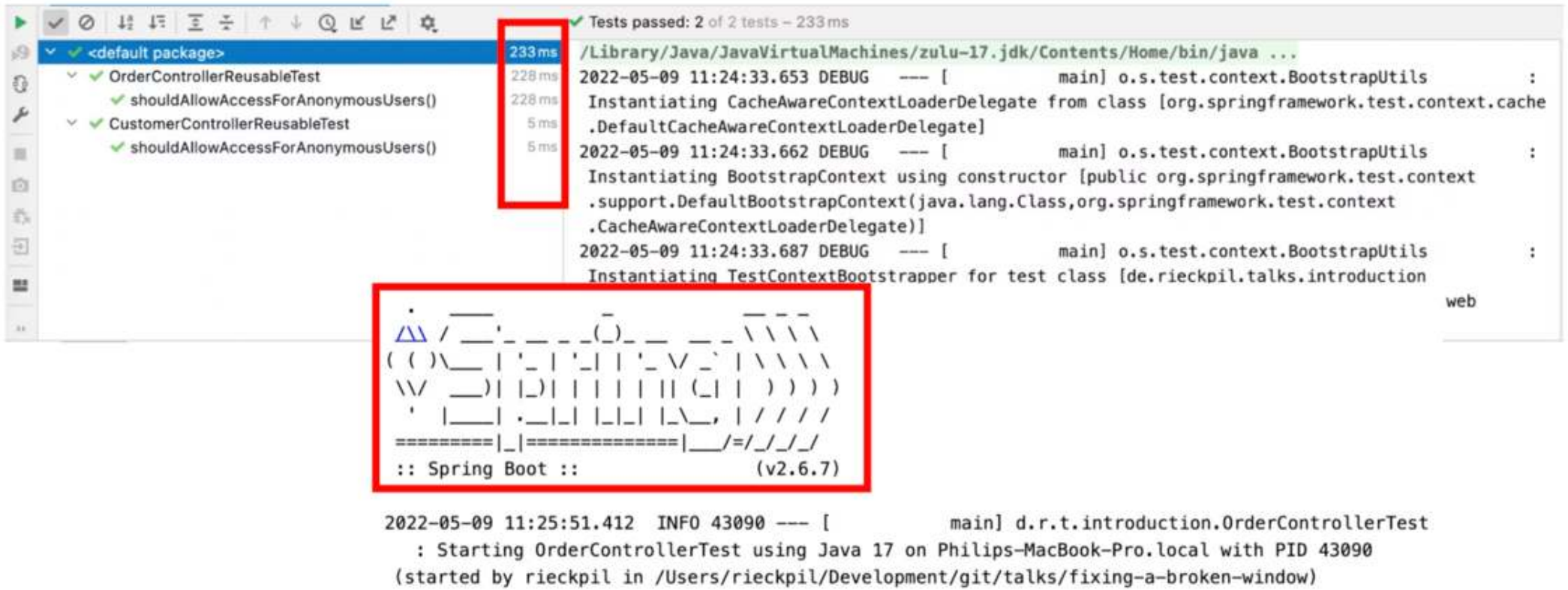
The following information is part of the Cache Key (`MergedContextConfiguration`):

- activeProfiles (`@ActiveProfiles`)
- contextInitializersClasses (`@ContextConfiguration`)
- propertySourceLocations (`@TestPropertySource`)
- propertySourceProperties (`@TestPropertySource`)
- contextCustomizer (`@MockitoBean` , `@MockBean` , `@DynamicPropertySource` , ...)
- etc.

```
Test class
    |
    ▼
MergedContextConfiguration(
  testClass, locations, classes,
  activeProfiles, propertyValues,
  contextInitializers, contextCustomizers   ← every @MockitoBean lands here
  ... etc.
)
    |
    ▼  hashCode() / equals()

Cache hit? → reuse context ✅
Cache miss? → start new context and store it 🆕
```

Detect Context Restarts - Visually



Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] org.springframework.test.context.cache   : Spring test
 ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
 hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] tractDirtiesContextTestExecutionListener : After test class:
 context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
  = [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
 '{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
 propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
 .SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
 .filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
 .DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```

Detect Context Restarts - with Tooling



An open-source Spring Test utility that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.

The Final Boss

Developers tend to consult AI/Stack Overflow (😓) for integration test issues and often copy advice from the internet without knowing the implications:

```
@SpringBootTest
@DirtiesContext
// this instructs Spring to remove the context from the cache
// and rebuild a new context on every request
public abstract class AbstractIntegrationTest {

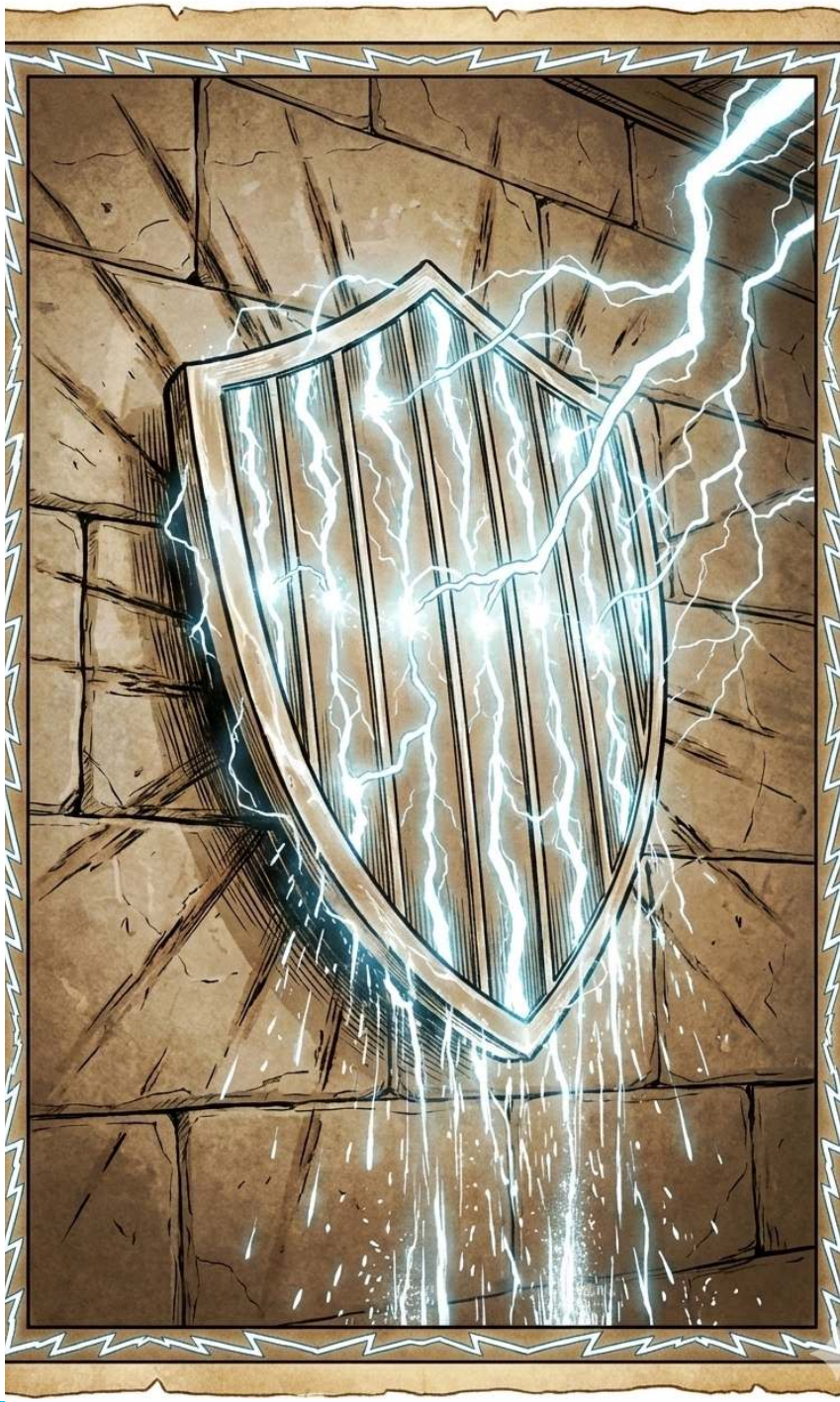
}
```

The setup above will **disable** the context caching feature and slow down the builds significantly!

QUEST ITEM 2

The Lightning Shield

Many Cores, One Goal



Test Parallelization

Goal: Reduce build time and get faster feedback

Requirements:

- No shared state
- No dependency between tests and their execution order
- No mutation of global state

Two ways to achieve this:

- Fork a new JVM with Surefire/Failsafe (or for the Gradle test task) and let it run in parallel
- Use JUnit Jupiter's parallelization mode and let it run in the same JVM with multiple threads



Java Test Parallelization Options

Maven Surefire Fork-Based

(Multiple JVMs)

JVM Fork 1

- Test Class A
- Test Class B
- Test Class C

JVM Fork 2

- Test Class D
- Test Class E
- Test Class F

Maven (forkCount) Features:

- Each fork is a separate JVM process
- Isolated memory spaces
- Higher memory overhead
- High isolation (one DB per fork)
- Configuration: forkCount, reuseForks

JUnit Jupiter Thread-Based

(Single JVM)

Single JVM

Thread Pool

Thread 1	Thread 2
Test Method A1	Test Method A2
Test Method B1	Test Method B2
Test Class C	Test Class D

JUnit Jupiter Features:

- Uses threads within a single JVM
- Shared memory space
- Lower memory overhead
- Configuration: junit.jupiter.execution.parallel.*

Test Parallelization 101

Using Surefire/Failsafe:

```xml
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <forkCount>1C</forkCount> <!-- 1 JVM per CPU core -->
  </configuration>
</plugin>
```

With JUnit Jupiter:

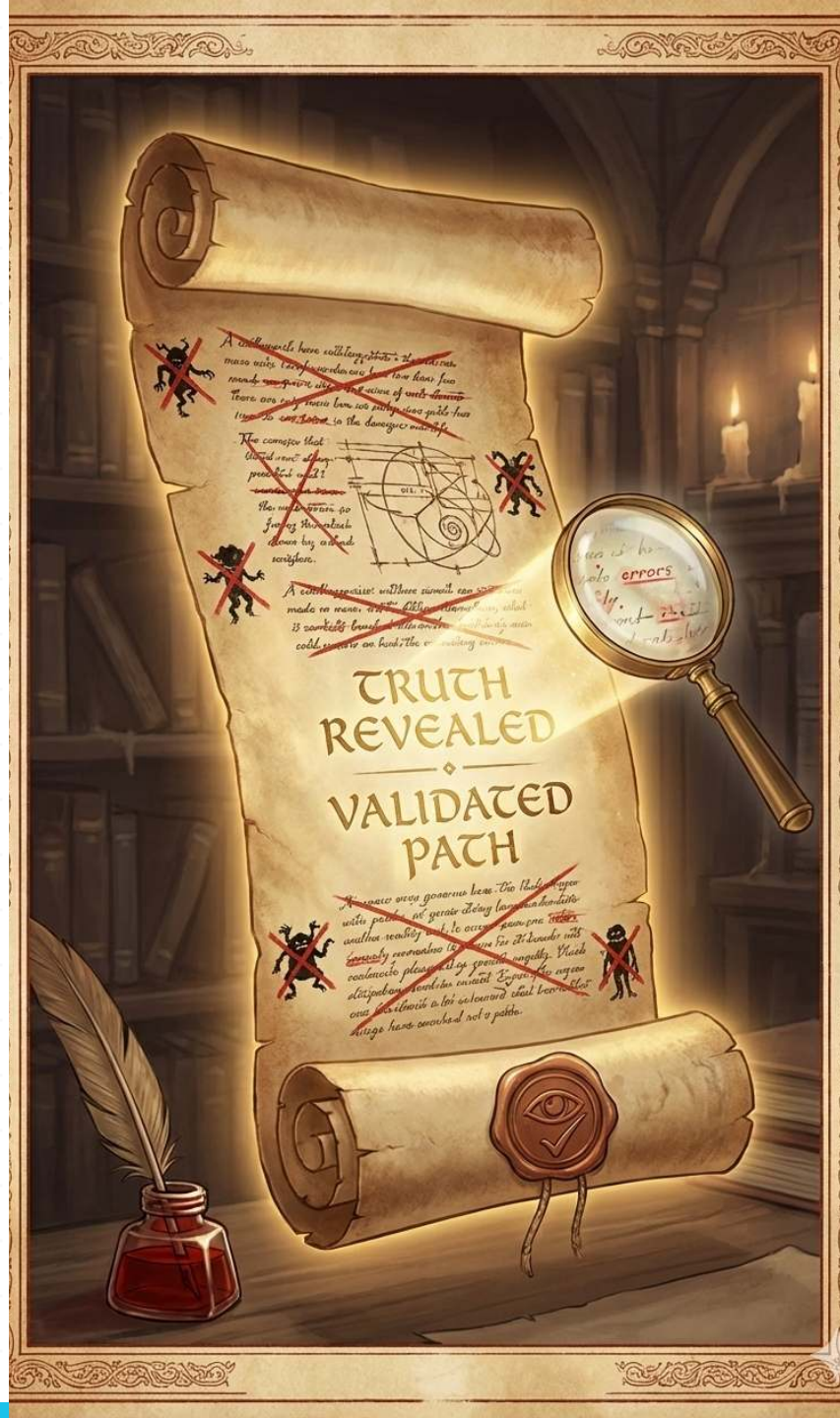
```
# src/test/resources/junit-platform.properties
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = same_thread
junit.jupiter.execution.parallel.mode.classes.default = concurrent
```



QUEST ITEM 3

The Scroll of Truth

Coverage Lies, Mutants Don't

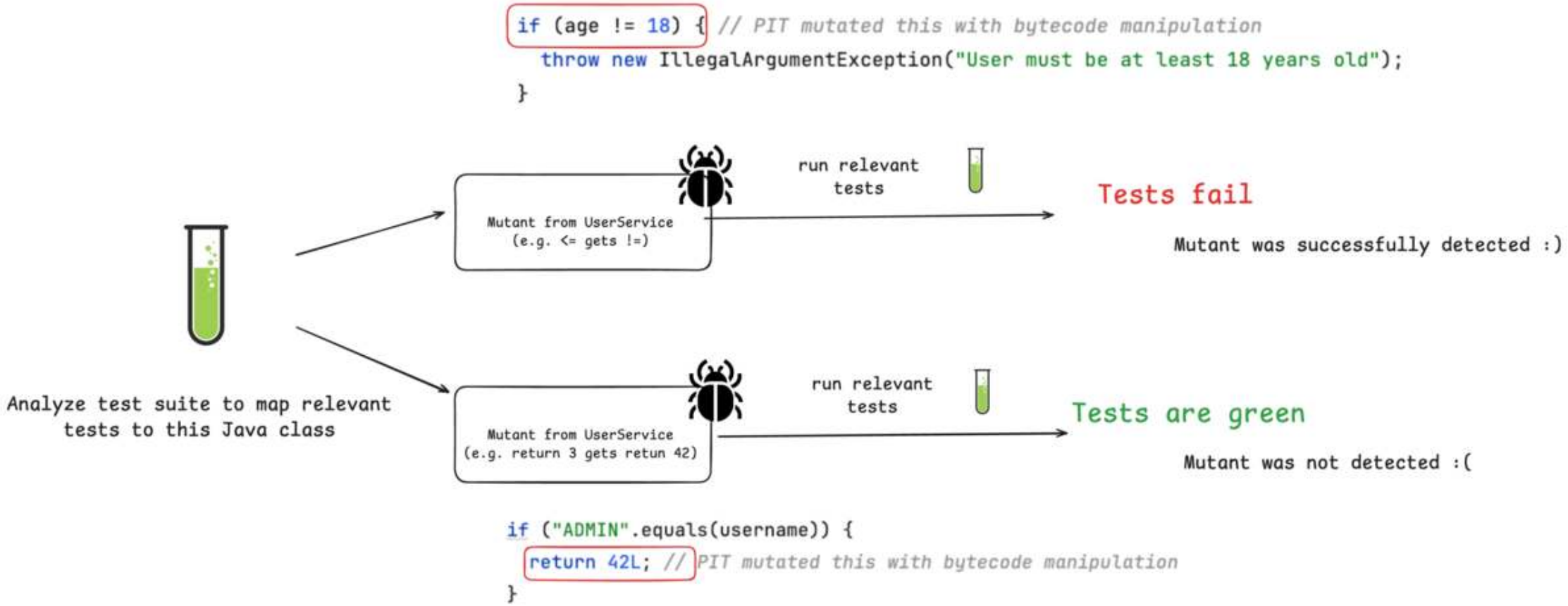


Let’s Challenge Code Coverage

Imagine a set of unit tests for this isolated business logic:

```
1   public Long registerUser(int age, String username) {
2
3     if (age <= 18) {
4       throw new IllegalArgumentException("User must be at least 18 years old");
5     }
6
7     if ("ADMIN".equalsIgnoreCase(username)) {
8       throw new IllegalArgumentException("Username 'ADMIN' is not allowed");
9     }
10
11    // ...
12
13  }
```

Idea: Introduce Regressions to Verify Test Quality



Introducing: Mutation Testing

- Having high code coverage might give you a **false sense of security**
- Mutation Testing with PIT
- Beyond Line Coverage: Traditional tools like JaCoCo show which code runs during tests, but PIT verifies if our tests actually detect when code behaves incorrectly by introducing "**mutations**" to our source code.
- Quality Guarantee: PIT automatically **modifies our code** (changing conditionals, return values, etc.) to ensure our tests fail when they should, **revealing blind spots** in seemingly comprehensive test suites.

Act 5: The Triumphant Exit

- Spring Boot applications come with batteries-included and excellent testing support
- We've completed three main quests: Unit testing, Sliced testing, and Integration testing
- Three core quest items help us to speed up and validate our tests:
  - Context Caching
  - Test Parallelization
  - Mutation Testing



Spring Boot 4 - Testing Support Keeps Improving

- **RestTestClient**: Modern, fluent alternative for the `TestRestTemplate` / `WebTestClient` / `RestAssured`.
- **Context pausing**: Cached test contexts are now automatically paused, eliminating resource conflicts from background processes.
- **JUnit 6**: Drop-in upgrade from JUnit 5 - far smoother than the JUnit 4 → 5 migration.
- **Testcontainers 2.0**: New `testcontainers-` prefix for modules, JUnit 4 support removed.
- **Bean overrides for non-singletons**: `@MockitoBean` and `@TestBean` now work with prototype and custom-scoped beans.

Making the Most of Agentic Development

- Invest in a **comprehensive** and **fast test suite**
- Reduce context switches for developers to stay in the loop
- Achieve **operational excellence**: monitoring, alerting, runbooks, A/B testing, feature flags, canary testing etc.
- Use e.g. Playwright MCP to let the AI self-verify features locally
- Provide custom skills to standardize code and tests across the company

Optimizing AI-written Tests with Skills

Get a **skillset** for fast & comprehensive tests, including a **test strategy** for the given project:

```
.claude/skills/
├── unit-testing              fast tests without context bloat
├── slice-testing             right-sized Spring context slices
├── slice-testing-webmvc      web layer with @WebMvcTest
├── integration-testing       full-context tests that stay fast
├── e2e-testing               user journeys against the running app
├── testcontainers-setup      real infrastructure, reused containers
└── test-setup-reviewer       flags test anti-patterns for you
```

Each skill includes rules, references, best-practices and antipatterns described as code and text.

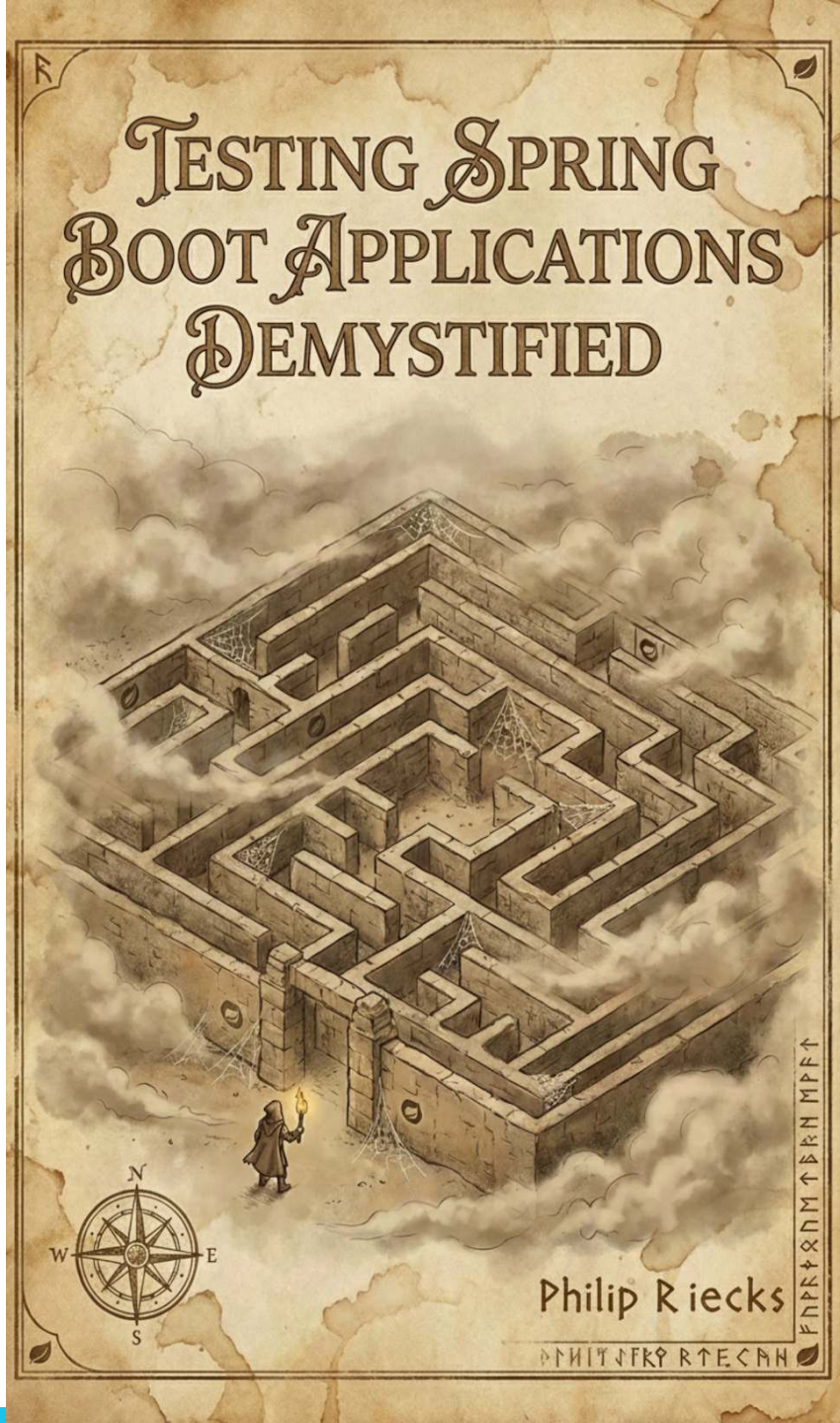
What Is a Skill? Example: `unit-testing`

```
.claude/skills/unit-testing/
├── SKILL.md                  name, description and the core workflow
├── testing-standards.md      naming, AssertJ, Arrange-Act-Assert
├── mocking-guidelines.md     when to mock and when not to
├── antipatterns.md           bad vs. good test examples
└── examples/
    └── OrderServiceTest.java
```

- `SKILL.md` has a short **description**, so the agent knows **when** to load the skill
- The other files are read **on demand**, so the context window stays small
- It is just **Markdown and code** in your repository - versioned, reviewed and shared like any other file

Don’t Leave Empty-Handed

- Get the complementary **Testing Spring Boot Applications Demystified** for free (instead of $9)
- 120+ Pages with practical hands-on advice to ship code with confidence
- Get the eBook by joining our newsletter



Joyful Testing!

Get your Spring Boot Testing eBook (120+ pages):

